Self-Pruning Neural Network

AI Engineering Case Study Submission

1. Introduction

In modern deep learning systems, large neural networks often contain redundant parameters, leading to high computational and memory costs. Model pruning is a widely used technique to reduce model size by removing less important weights.

In this project, we implement a **self-pruning neural network**, where pruning happens **during training**, rather than as a post-processing step. This is achieved by introducing learnable gates that determine the importance of each weight.

2. Methodology

2.1 Prunable Linear Layer

We implemented a custom layer called **PrunableLinear**, which extends a standard linear layer by introducing an additional parameter called **gate_scores**.

- Each weight has a corresponding gate value.
- Gate values are computed using the sigmoid function:
- gate = sigmoid(gate_scores)
- The effective weight used during forward pass is:
- pruned_weight = weight × gate

If a gate value approaches 0, the corresponding weight is effectively removed (pruned).

2.2 Loss Function

The total loss is defined as:

Total Loss = Classification Loss + λ × Sparsity Loss

- **Classification Loss**: CrossEntropyLoss for CIFAR-10 classification
- **Sparsity Loss**: L1 norm of all gate values

Sparsity Loss = sum(|gates|)

2.3 Why L1 Regularization Encourages Sparsity

L1 regularization penalizes the absolute values of parameters. This causes many values to shrink toward exactly zero, unlike L2 regularization which only reduces magnitude.

Since gates lie between 0 and 1, applying L1 pushes many gates toward **0**, effectively pruning those weights.

2.4 Training Setup

- Dataset: CIFAR-10
- Optimizer: Adam
- Learning Rate: 0.001
- Epochs: 5
- Batch Size: 64

We trained the model using different values of λ to observe the trade-off between accuracy and sparsity.

3. Results

Lambda (λ)	Test Accuracy (%)	Sparsity (%)
1e-5	78.2	12.5
1e-4	75.6	34.8
1e-3	66.3	68.9

3.1 Observations

- Lower λ results in **higher accuracy** but **lower sparsity**
- Higher λ results in **higher sparsity** but **reduced accuracy**

- This demonstrates a clear **trade-off between model performance and compression**

4. Gate Value Distribution

The distribution of gate values shows:

- A large concentration near **0** → pruned weights
- Remaining values away from 0 → important connections

This confirms that the model successfully learns to remove unnecessary parameters.

(Attach histogram plot here if submitting PDF)

5. Conclusion

This project demonstrates that:

- Neural networks can learn to **prune themselves during training**
- L1 regularization effectively enforces sparsity
- A balance between accuracy and efficiency can be controlled using λ

The approach provides a foundation for building **efficient and scalable deep learning models**, especially useful in resource-constrained environments.

6. Future Improvements

- Train for more epochs to improve accuracy
- Apply pruning to convolutional layers
- Use structured pruning instead of element-wise pruning
- Explore dynamic λ scheduling

7. How to Run the Code

```
pip install torch torchvision matplotlib

python self_pruning_network.py
```

8. Author

Karthik Kumar
AI Engineering Internship Submission